🧱 Section 6 — Building an Image (Dockerfile Basics)

This section introduces Docker images, Dockerfiles, and how they relate to OS concepts such as layered filesystems and process startup.

6.1 What Is a Docker Image?

A **Docker image** is:

- A **read-only snapshot** of:
  - a root filesystem
  - installed packages and libraries
  - configuration files
  - default commands/entrypoints
- Composed of multiple **layers**:
  - Each layer represents changes from the layer below (e.g., new files, modified packages).
  - Layers are typically implemented via **union/overlay filesystems** (e.g., OverlayFS).

OS concept:

6.2 Containers vs. Images

- **Image**:
  - Static template describing what a filesystem and entrypoint should look like.
  - Read-only layers.
- **Container**:
  - A **running instance** of an image.
  - Adds a **writable layer** on top of the image's read-only layers.

Analogy (technical):

- Image → template of a root filesystem.
- Container → running process tree using that filesystem with namespaces + cgroups applied.

6.3 The Role of a Dockerfile

A **Dockerfile** is a text file that:

- Describes **how to build** an image.
- Contains a sequence of instructions:
  - `FROM`, `RUN`, `COPY`, `WORKDIR`, `CMD`, `ENTRYPOINT`, etc.
- Each instruction creates a new **layer** in the resulting image (with some exceptions in newer optimizations).

Building an image:

```
docker build -t my-image-name .
```

- `-t my-image-name` gives the resulting image a tag.
- `.` points to the **build context** (current directory, including the Dockerfile).

6.4 Minimal Example: A Simple Ubuntu-Based Image

Dockerfile:

```
# 1) Base image (defines initial root filesystem + environment)
FROM ubuntu:22.04

# 2) Install packages (creates a new layer)
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        python3 \
        python3-pip && \
    rm -rf /var/lib/apt/lists/*

# 3) Set working directory inside the image
WORKDIR /app

# 4) Copy application code into the image
COPY app.py /app/app.py

# 5) Define the default command when the container runs
```

6.5 OS View: What Happens When a Container from This Image Runs?

1. The Docker engine:
    - Constructs a unified view of all image layers using an overlay filesystem.
    - Creates the container’s root filesystem from this stack.
2. The kernel:
    - Sets up namespaces (PID, mount, network, etc.).
    - Attaches the container’s processes to relevant cgroups (CPU, memory, etc.).
3. The entry command:
    - `CMD ["python3", "app.py"]` becomes the initial process (PID 1) in the container’s PID namespace.
    - This process runs with the container’s isolated filesystem, network, and other

6.6 Common Dockerfile Instructions (Conceptual Overview)

Instruction	Purpose	OS/FS perspective
`FROM`	Select base image	Defines initial root filesystem and environment
`RUN`	Execute command at build time	Modifies the filesystem; creates a new read-only layer
`COPY`	Copy files from host into image	Adds files into the image's filesystem
`ADD`	Like `COPY`, but can also fetch/untar	Same as `COPY` + extra behavior
`WORKDIR`	Set default working directory	Changes effective `cwd` for subsequent instructions and container start

6.7 Example: OS-Focused Image for Observing Processes

Goal: Create an image that starts a small monitoring loop printing process and memory info.

Dockerfile:

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        procps \
        python3 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /mon

COPY monitor.py /mon/monitor.py

CMD ["python3", "monitor.py"]
```

6.8 Image Layers and Caching Behavior (Brief)

- Each line like `RUN`, `COPY`, `ADD` produces a new **layer**.
- Docker uses **layer caching**:
  - If the instruction and its context are unchanged, Docker reuses the existing layer.
- Order matters:
  - Frequently changing instructions (e.g., `COPY . /app`) should come **later**.
  - Stable instructions (e.g., `RUN apt-get install ...`) should come **earlier**.

OS concept:

> Layered filesystems use copy-on-write techniques to minimize duplication; caching exploits immutability of layers.

6.9 Summary of Commands Used in Section 6

These can be placed in a slide or appendix as a quick reference.

```
# Build an image from a Dockerfile in the current directory
docker build -t example-python-app .

# Run a container from the built image
docker run --rm example-python-app

# Build the monitoring example image
docker build -t monitor-container .

# Run the monitoring container interactively
docker run --rm -it monitor-container
```

(From earlier sections, typical supporting commands include `docker run -it ubuntu bash`, `docker ps`, `docker stop`, `docker rm`, etc.)

If desired, the next section can focus on **Volumes and the Filesystem Model** or **Container Networking**, continuing in the same style and adding another command summary at the end.